

Relatório 3 - Processos e Threads

1. Enunciado

Análise e execute os códigos disponibilizados e explique o comportamento observado nos valores de saída da variável contador. Discuta o que acontece em cada código, identificando possíveis problemas e apresentando uma solução adequada para cada caso.

Em seguida, discuta os conceitos de processo e thread, destacando as principais diferenças entre eles, especialmente em relação ao compartilhamento de memória e à execução concorrente.

2. Objetivo da Tarefa

O objetivo desta atividade é compreender as diferenças fundamentais entre a execução concorrente via **Processos** (com espaço de endereçamento isolado) e via **Threads** (com espaço de endereçamento compartilhado). Busca-se analisar o problema da **Condição de Corrida (Race Condition)** em variáveis compartilhadas e demonstrar como utilizar mecanismos de sincronização adequados, como **Semáforos POSIX** (para processos) e **Mutexes** (para threads).

3. Metodologia

A solução foi dividida e analisada sob duas abordagens de programação concorrente em C (POSIX):

1. **Abordagem via Processos (cont1.c):** Utiliza a chamada de sistema `fork()` para criar 4 processos filhos. Como os processos possuem memória isolada por padrão, foi utilizada a função `mmap()` com a *flag* `MAP_SHARED | MAP_ANONYMOUS` para criar um segmento de memória RAM verdadeiramente compartilhado para a variável `contador`. A sincronização de acesso a essa região entre processos independentes foi garantida por meio de um **Semáforo Nomeado POSIX** (`sem_open`, `sem_wait`, `sem_post`).
2. **Abordagem via Threads (cont2.c):** Utiliza a biblioteca `pthread` (`pthread_create`, `pthread_join`) para criar 4 threads dentro do mesmo processo. As threads compartilham naturalmente a memória global do processo (variável `int contador = 0;`). A exclusão mútua foi garantida utilizando uma trava de **Mutex POSIX** (`pthread_mutex_lock`, `pthread_mutex_unlock`).

Ambos os programas foram executados no macOS para realizar 4.000.000 de incrementos totais (4 entidades × 1.000.000 incrementos cada), avaliando a correteza do resultado final.

4. Código-Fonte

cont1.c (Processos com Memória Compartilhada e Semáforo)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <fcntl.h> // Configurações do sem_open

#define NUM_PROCESSOS 4
#define INCREMENTOS 1000000

int main() {
    // Aloca a memória compartilhada APENAS para o contador
    int *contador = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0
```

```

if (contador == MAP_FAILED) {
    perror("Erro na memória compartilhada");
    exit(EXIT_FAILURE);
}

*contador = 0; // Inicializa o contador com zero

// Limpa o semáforo caso o programa tenha parado de forma inesperada antes
sem_unlink("/meu_semaforo");

// Cria o semáforo. Inicializado com 1 (binário/destravado)
sem_t *semaforo = sem_open("/meu_semaforo", O_CREAT, 0644, 1);

printf("Contador inicial: %d\n", *contador);

for (int i = 0; i < NUM_PROCESSOS; i++) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("Erro ao criar processo");
        exit(EXIT_FAILURE);
    }
    if (pid == 0) {
        // Código executado pelo processo filho
        for (int j = 0; j < INCREMENTOS; j++) {
            sem_wait(semaforo); // Trava (Decrementa o semáforo)
            (*contador)++;      // Região Crítica: Incremento seguro
            sem_post(semaforo); // Destrava (Incrementa o semáforo)
        }
        printf("Filho %d terminou. Contador atual = %d\n", getpid(), *contador);
        exit(EXIT_SUCCESS);
    }
}

// Processo pai espera todos os filhos terminarem
for (int i = 0; i < NUM_PROCESSOS; i++) {
    wait(NULL);
}

printf("\nContador no processo pai: %d\n", *contador);

// Limpeza de recursos
sem_close(semaforo);
sem_unlink("/meu_semaforo");
munmap(contador, sizeof(int));

return 0;
}

```

cont2.c (Threads com Mutex)

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

```

```

#define NUM_THREADS 4
#define INCREMENTOS 1000000

// compilar: gcc cont2.c -o cont2 -pthread

int contador = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *incrementar(void *arg) {
    for (int i = 0; i < INCREMENTOS; i++) {
        pthread_mutex_lock(&mutex);    // Trava: apenas uma thread passa por vez
        contador++;
        pthread_mutex_unlock(&mutex); // Destrava: libera para a próxima thread
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    printf("Contador inicial: %d\n", contador);

    // Criar as threads
    for (int i = 0; i < NUM_THREADS; i++) {
        int erro = pthread_create(
            &threads[i],
            NULL,
            incrementar,
            NULL);
        if (erro != 0) {
            fprintf(stderr, "Erro ao criar thread\n");
            exit(EXIT_FAILURE);
        }
    }

    // Esperar todas as threads
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\nContador final: %d\n", contador);

    pthread_mutex_destroy(&mutex);

    return 0;
}

```

5. Resultados

5.1. Saída Observada nos Executáveis Sincronizados

Tanto o programa `cont1` (Processos) quanto o programa `cont2` (Threads) executaram com sucesso e produziram o valor matemático exato esperado ao final:

```
Contador inicial: 0
Filho 84102 terminou. Contador atual = 3849120
Filho 84103 terminou. Contador atual = 3912040
Filho 84104 terminou. Contador atual = 3984100
Filho 84105 terminou. Contador atual = 4000000
```

Contador no processo pai: 4000000

(Saída correspondente ao cont1.c)

```
Contador inicial: 0
```

```
Contador final: 4000000
```

(Saída correspondente ao cont2.c)

5.2. A Condição de Corrida (Sem Sincronização)

Sem as travas (`sem_wait/mutex`), o resultado final fica inconsistente e muito menor que o esperado devido à não-atomicidade do incremento `contador++` no hardware, que se divide em três passos: **Read** (Lê da RAM), **Modify** (Soma 1) e **Write** (Escreve na RAM).

Quando duas threads/processos leem o mesmo valor simultaneamente antes que a outra grave a atualização, ambas gravam o mesmo resultado. Isso faz com que múltiplos incrementos sejam sobrepostos e perdidos na memória RAM.

5.3. Análise dos Mecanismos de Solução

1. Mapeamento de Memória e Semáforos (`cont1.c`):

- Ao executar um `fork()`, o sistema operacional duplica a memória do processo pai via *Copy-on-Write* (COW). Sem o `mmap` com a flag `MAP_SHARED`, os processos filhos alterariam apenas suas cópias privadas da variável. A função `mmap()` força o Kernel a mapear a mesma página física de RAM para todos os processos.
- Para proteger essa memória compartilhada contra condições de corrida, utilizou-se um semáforo nomeado POSIX (`sem_open`). O comando `sem_wait` trava a região crítica e `sem_post` a libera para o próximo processo.

2. Mutex em Threads (`cont2.c`):

- Como as threads pertencem ao mesmo processo, elas já compartilham a mesma memória RAM por padrão (seção de dados globais e *Heap*). Não é necessário usar `mmap()`.
- Utilizou-se o `pthread_mutex_t`, que é uma trava leve otimizada para garantir que apenas uma thread por vez execute a instrução `contador++`.

5.4. Comparativo Teórico: Processo vs. Thread

- **Espaço de Memória:**
 - **Processos:** Possuem espaço de endereçamento totalmente isolado pelo sistema operacional. Para compartilhar dados, exigem o uso explícito de mecanismos de IPC (como `mmap`, memória compartilhada ou *pipes*).
 - **Threads:** Compartilham naturalmente o mesmo espaço de memória do processo pai (variáveis globais e *Heap*). Cada thread possui apenas sua própria Pilha (*Stack*) e registradores.
- **Custo de Criação e Troca de Contexto:**
 - **Processos:** Alto custo para o sistema operacional, pois exige a alocação de novas tabelas de páginas e atualização da TLB (*Translation Lookaside Buffer*).
 - **Threads:** Baixo custo (*Lightweight Processes*), pois compartilham a mesma tabela de páginas, exigindo apenas a troca de contexto dos registradores e ponteiro de pilha.
- **Tolerância a Falhas:**

- **Processos:** Alta isolamento. Se um processo filho falhar (ex: *Segmentation Fault*), os demais processos continuam executando normalmente.
- **Threads:** Baixa tolerância. Como compartilham o mesmo espaço de memória, se uma única thread causar uma falha grave, o processo inteiro é encerrado.
- **Mecanismos de Sincronização:**
 - **Processos:** Utilizam Semáforos (nomeados ou em memória compartilhada), *Pipes*, Sockets ou Sinais do sistema operacional.
 - **Threads:** Utilizam Mutexes, Variáveis de Condição ou Semáforos nativos da biblioteca de threads.

6. Conclusão

A atividade demonstra na prática as diferenças de arquitetura entre processos e threads nos sistemas baseados em POSIX. Enquanto os processos oferecem alto isolamento ao custo de maior sobrecarga e necessidade de mecanismos explícitos para compartilhar memória (**mmap**), as threads compartilham memória nativamente, tornando a comunicação mais rápida. Em ambos os casos de execução concorrente, o uso de primitivas de sincronização (Semáforos e Mutexes) é indispensável para evitar condições de corrida na região crítica e garantir a integridade dos dados.